

TECHNICAL INVESTIGATION, ROS 2 TARGET-POSE VALIDATION AND SIMULATION INTEGRATION SUPPORT

PROJECT 7 – E-WASTE BATTERY EXTRACTION SYSTEM

SYSTEMS AND SIMULATIONS TEAM

SIT378 – CAPSTONE TEAM PROJECT (B)

STUDENT: RAMNEEK SINGH BHANGU

**TECHNICAL CONTRIBUTION: RESEARCH, SIMULATION STRATEGY, ROS 2
TARGET VALIDATION AND INTEGRATION SUPPORT**

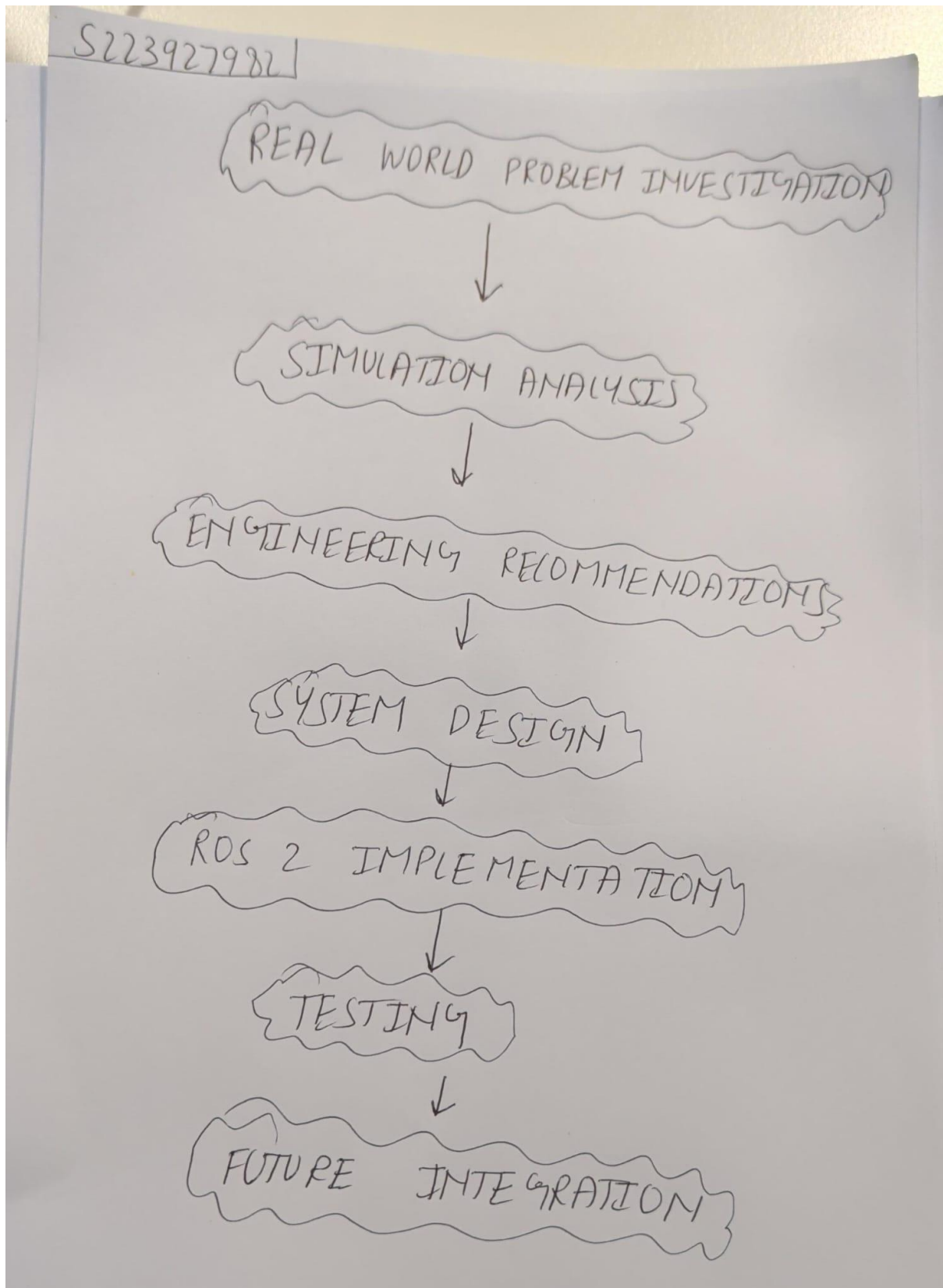
STATUS: ONGOING IMPLEMENTATION

1. INTRODUCTION

Project 7 – E-Waste Battery Extraction System aims to develop an intelligent robotic solution capable of identifying, accessing, extracting and sorting lithium-ion batteries from electronic waste. The project requires close integration between Computer Vision, Simulation and Robotics, with each subsystem contributing a different part of the complete autonomous workflow.

My technical contribution has progressed through two main stages. During Sprint 1, I focused on research, systems analysis and engineering design to understand the practical challenges associated with realistic smartphone battery extraction and determine how those challenges could be represented within the project's simulation environment. During Sprint 2, I began translating these findings into practical software implementation by developing a ROS 2 target-pose validation component intended to operate between perception output and robotic motion planning.

This work therefore follows a complete engineering progression:



The overall aim is to produce a reliable and reusable integration component that improves the hand-off between Computer Vision, Simulation and Robotics.

2. BACKGROUND AND MOTIVATION

The existing simulation environment successfully demonstrates simplified robotic battery handling, where a robotic arm can collect a loose battery and place it into a sorting location. However, real smartphone battery extraction involves significantly greater complexity.

My initial investigation established that batteries are rarely available as loose objects. Instead, they are normally enclosed within the device and may require several operations before removal is possible, including enclosure opening, connector disconnection, adhesive release and controlled lifting.

The investigation also showed that smartphone conditions vary significantly. Batteries may be secured with adhesive strips, enclosed behind waterproof covers, connected through ribbon cables, swollen, damaged or affected by contamination. These variations mean that realistic extraction cannot be represented as a single fixed robotic movement.

This motivated a broader investigation into how the project should approach realistic simulation and future integration.

3. SPRINT 1 – TECHNICAL RESEARCH AND ENGINEERING INVESTIGATION

3.1 REALISTIC SMARTPHONE BATTERY EXTRACTION

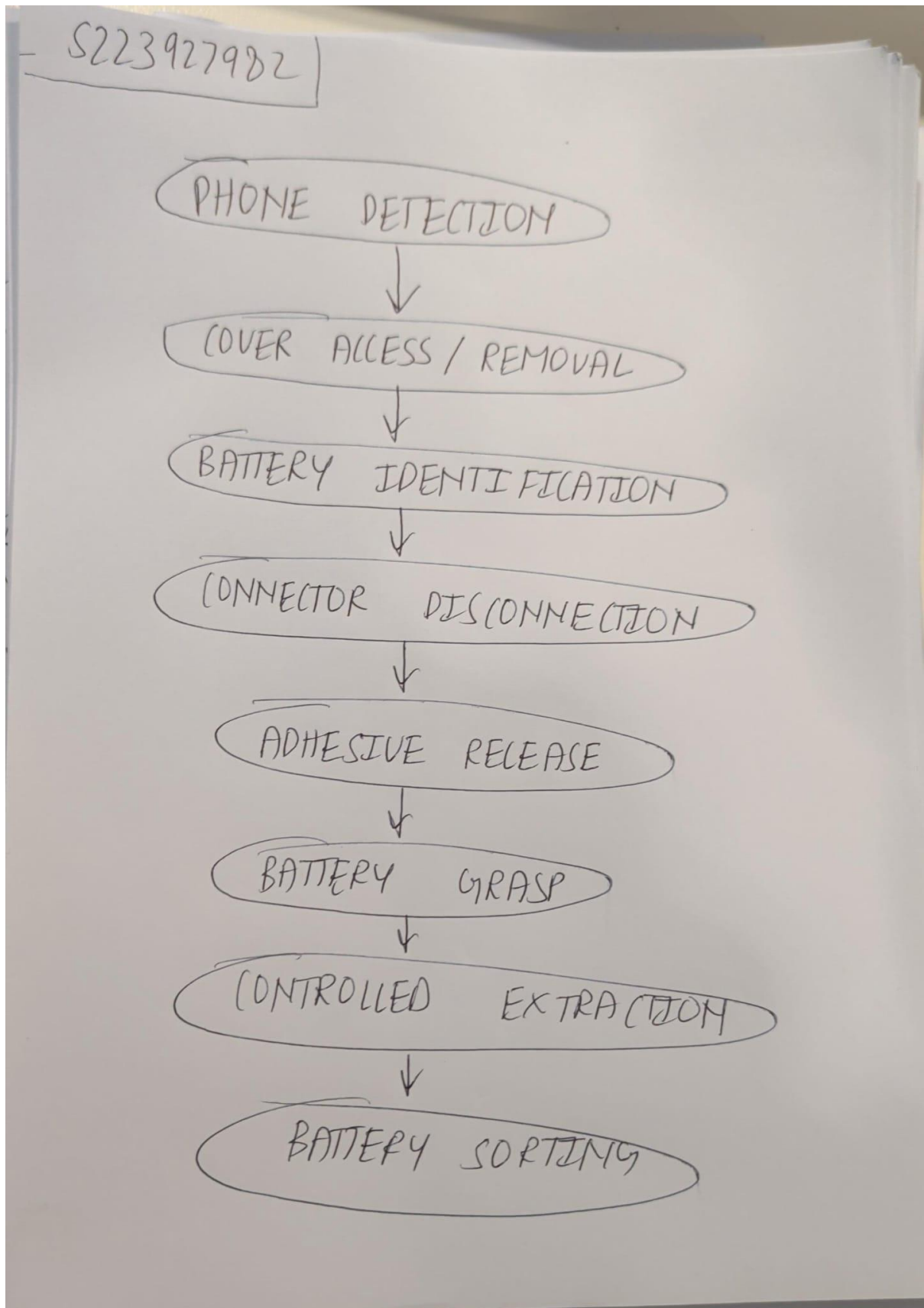
The first stage of my contribution involved researching how batteries are actually removed from modern smartphones.

The investigation included:

- adhesive-secured batteries;
- stretch-release adhesive;
- waterproof enclosures;
- removable and press-fit covers;
- ribbon cables and connectors;
- swollen batteries;
- damaged devices;
- water-damaged phones;
- protective cases.

A key finding was that battery extraction should be represented as a **multi-stage process** rather than a basic pick-and-place task.

A more realistic extraction workflow is:



This finding changed the way the simulation problem should be approached.

4. EXTRACTION-METHOD MATRIX

To make the research directly useful for future implementation, I developed an extraction-method matrix comparing different device conditions with potential Gazebo representations.

The matrix considered situations including:

- loose batteries;
- adhesive-secured batteries;
- stretch-release adhesive;
- strong adhesive;
- rear-cover removal;
- ribbon cable connection;
- swollen batteries;
- wet devices;
- damaged covers;
- protective cases.

For each condition, I evaluated:

1. the real-world engineering challenge;
2. the recommended simulation representation;
3. the appropriate robotic response.

The purpose of this matrix was to reduce uncertainty during future implementation and prevent the team from attempting unnecessarily complex physical simulation where simpler engineering abstractions would be more practical.

5. GAZEBO PHYSICS INVESTIGATION

The second major part of my research investigated whether Gazebo could realistically reproduce the physical behaviours involved in battery extraction.

Gazebo provides strong support for:

- rigid-body dynamics;
- gravity;
- inertia;
- joint constraints;
- collision detection;
- friction;

- robot simulation;
- ROS 2 integration.

These capabilities make it highly suitable for robotic manipulation and general simulation work.

However, the investigation identified limitations in several areas.

5.1 ADHESIVE BEHAVIOUR

Real battery adhesive can stretch, peel, deform and fail depending on applied force and direction.

Standard Gazebo physics does not realistically reproduce this behaviour.

Instead of attempting complex adhesive deformation, I recommended that the battery remain constrained until predefined release conditions are met.

Possible conditions include:

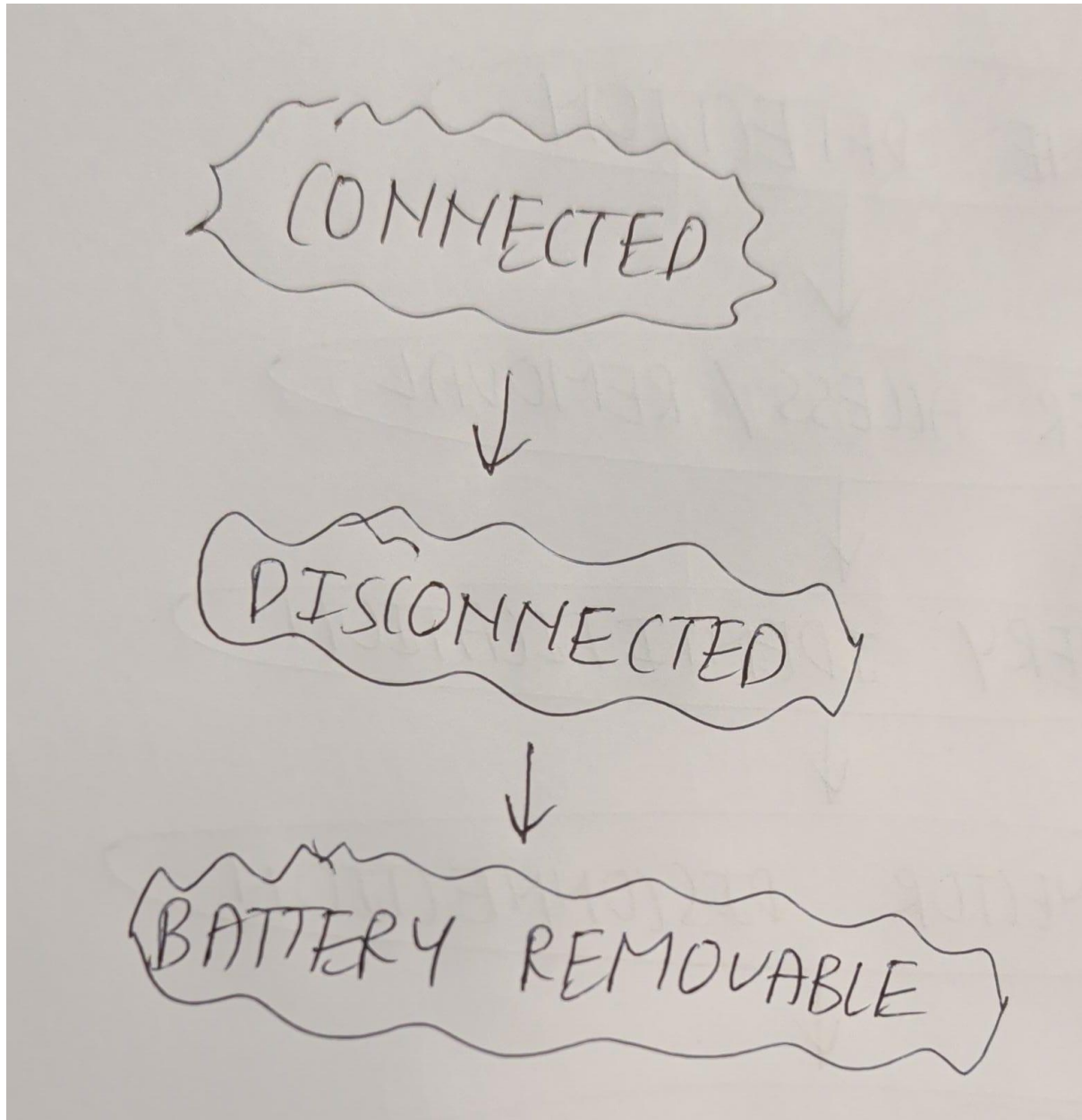
- correct tool alignment;
- successful suction attachment;
- minimum extraction force;
- elapsed time;
- connector removal;
- software-triggered release.

Once the required conditions are satisfied, the constraint can be removed programmatically.

5.2 FLEXIBLE COMPONENTS

Ribbon cables and flexible connectors are difficult to model accurately using conventional rigid-body simulation.

I therefore recommended representing them using logical states such as:



This maintains realistic process logic without requiring complex flexible-body simulation.

5.3 SUCTION MANIPULATION

Real suction depends on pressure, leakage, surface contact and seal quality.

Instead of attempting detailed vacuum simulation, the recommended approach is to use controlled temporary attachment when:

- contact is detected;
- alignment is appropriate;
- suction has been commanded.

The attachment can then be removed when the battery is released.

6. HYBRID SIMULATION STRATEGY

Based on these findings, I proposed a **hybrid simulation strategy**.

The principle is:

Use physics where physics provides meaningful engineering behaviour, and use scripted logical behaviour where high-fidelity physical simulation adds excessive complexity without sufficient project value.

Under this approach:

Rigid-body simulation is used for:

- robot links;
- batteries;
- phone models;
- work surfaces;
- gravity;
- inertia;
- joint motion;
- collisions.

Scripted/logical behaviour is used for:

- adhesive release;
- ribbon-cable states;
- suction attachment;
- condition-specific extraction logic;
- damaged-device scenarios.

This provides a balance between realism, computational efficiency and development feasibility.

7. MOVEIT! 2 MOTION-PLANNING INVESTIGATION

Another major part of my research involved evaluating motion-planning approaches suitable for robotic battery extraction.

The current project workflow relies heavily on predefined movement sequences. While this is suitable for demonstrations, realistic battery extraction requires adaptive, collision-aware motion.

I evaluated several MoveIt! planners:

- OMPL;
- CHOMP;
- STOMP;
- Pilz Industrial Motion Planner.

7.1 OMPL

OMPL provides several sampling-based planners including:

- RRTConnect;
- RRT*;
- PRM;
- EST.

It offers strong collision-aware planning and good computational performance.

I recommended **OMPL using RRTConnect** as the most appropriate initial planner for the project because it provides a strong balance between:

- planning speed;
- reliability;
- ROS 2 compatibility;
- obstacle avoidance;
- implementation maturity.

7.2 ALTERNATIVE PLANNERS

PILZ

Useful for straight-line industrial motions such as controlled lifting.

CHOMP

Provides smoother trajectories but requires greater computational effort.

STOMP

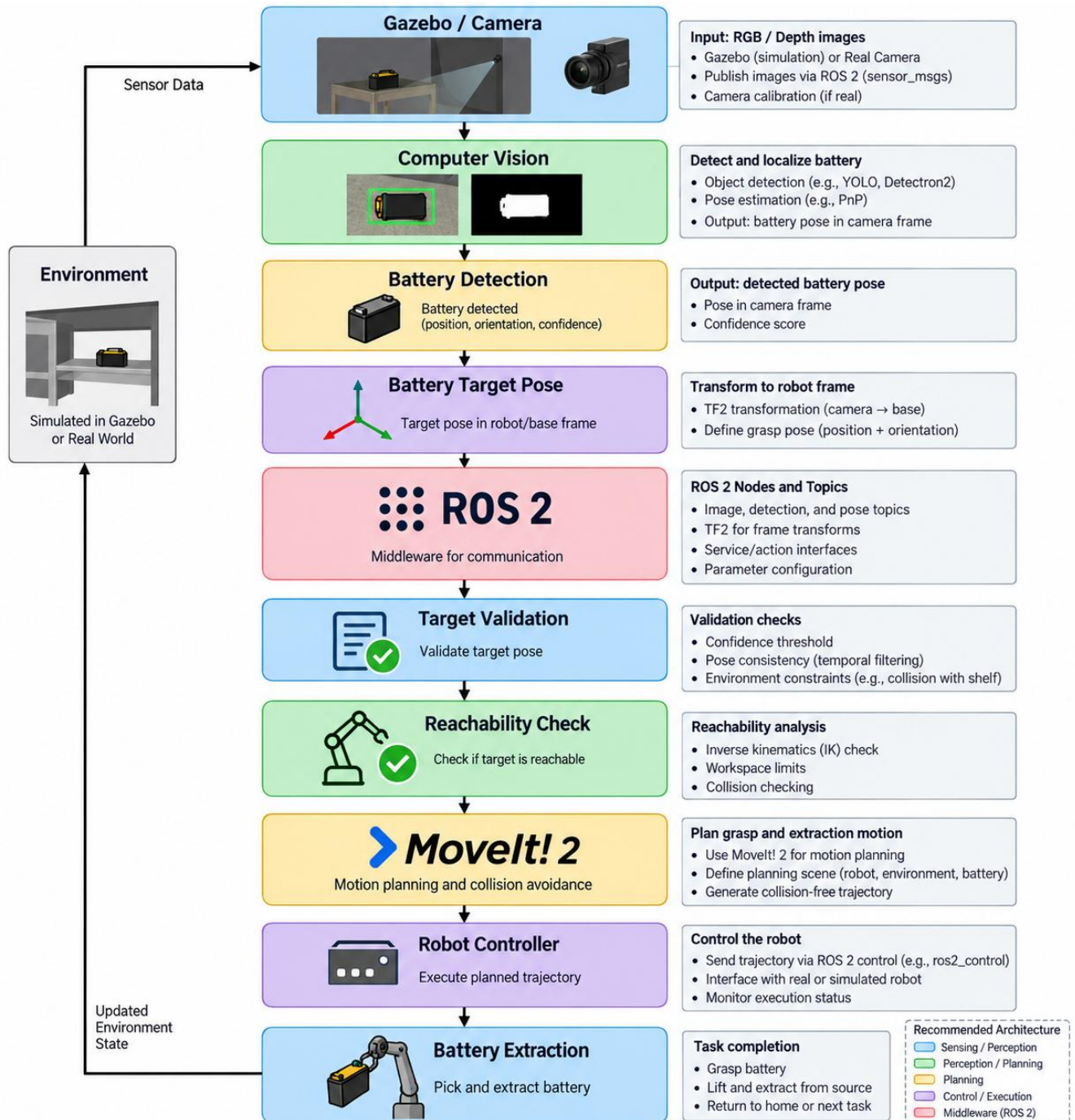
Useful in complex environments but introduces additional complexity.

My recommendation was therefore to use OMPL first and investigate optimisation planners later once the integration pipeline becomes stable.

8. PROPOSED SYSTEM ARCHITECTURE

The research also considered how the Simulation, Computer Vision and Robotics teams should interact.

The recommended architecture is:



ROS 2 provides the communication layer between these components.

This modular architecture enables the different sub-teams to develop independently while maintaining defined system interfaces.

9. IMPORTANT INTEGRATION RISK IDENTIFIED

One of the key findings from the Sprint 1 research concerned **coordinate accuracy and subsystem integration**.

Computer Vision may detect a battery successfully, but the target coordinates still need to be:

- expressed in the correct coordinate frame;
- represented using appropriate units;
- numerically valid;
- located within the simulation workspace;
- physically reachable by the robotic manipulator.

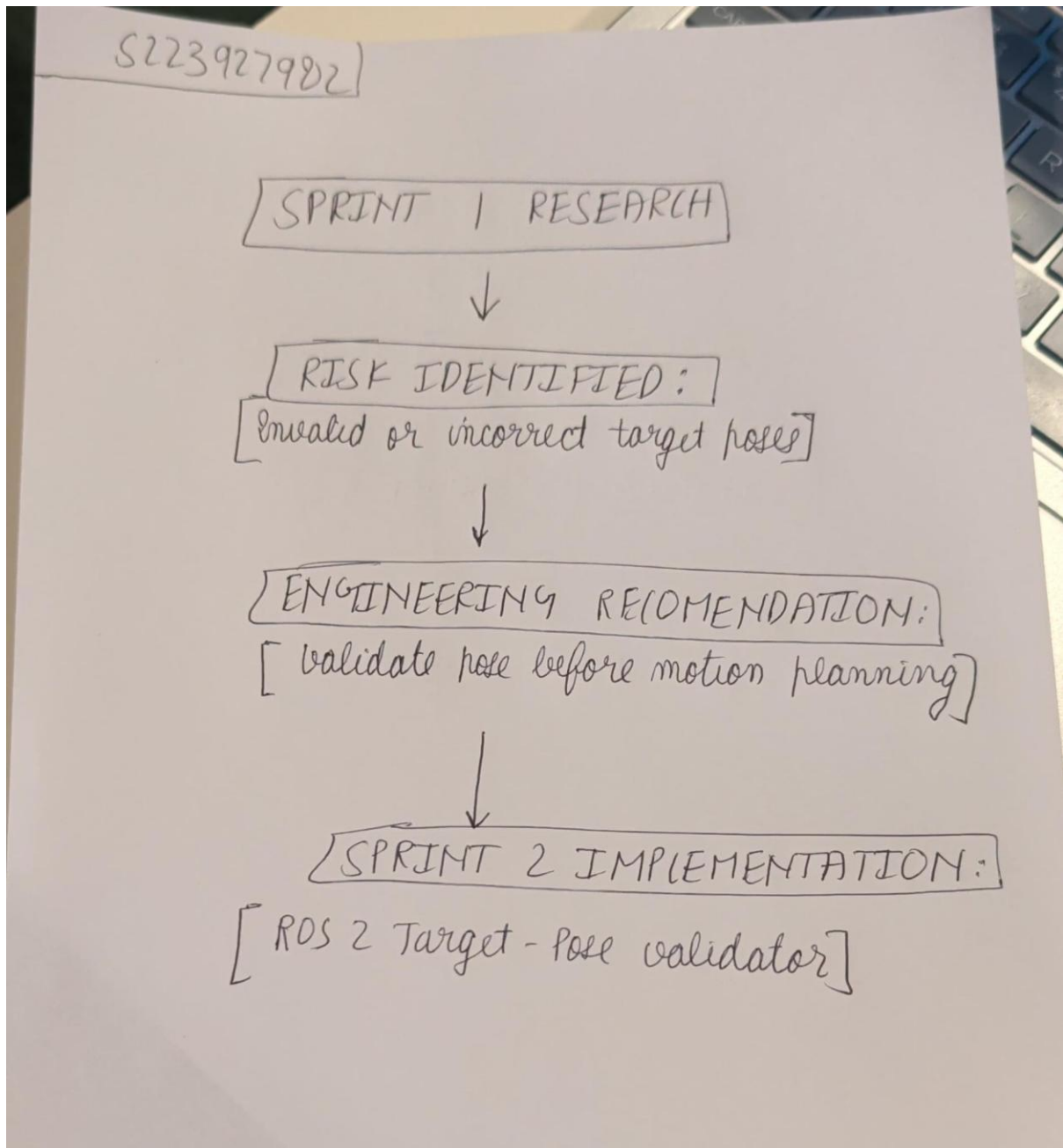
The investigation identified coordinate transformation, localisation accuracy and subsystem compatibility as important integration risks.

This finding directly motivated my Sprint 2 implementation.

10. TRANSITION FROM RESEARCH TO IMPLEMENTATION

Rather than treating Sprint 2 as an unrelated coding activity, I selected an implementation task that directly translates one of the Sprint 1 recommendations into software.

The progression is:



This demonstrates progression from engineering analysis to practical implementation.

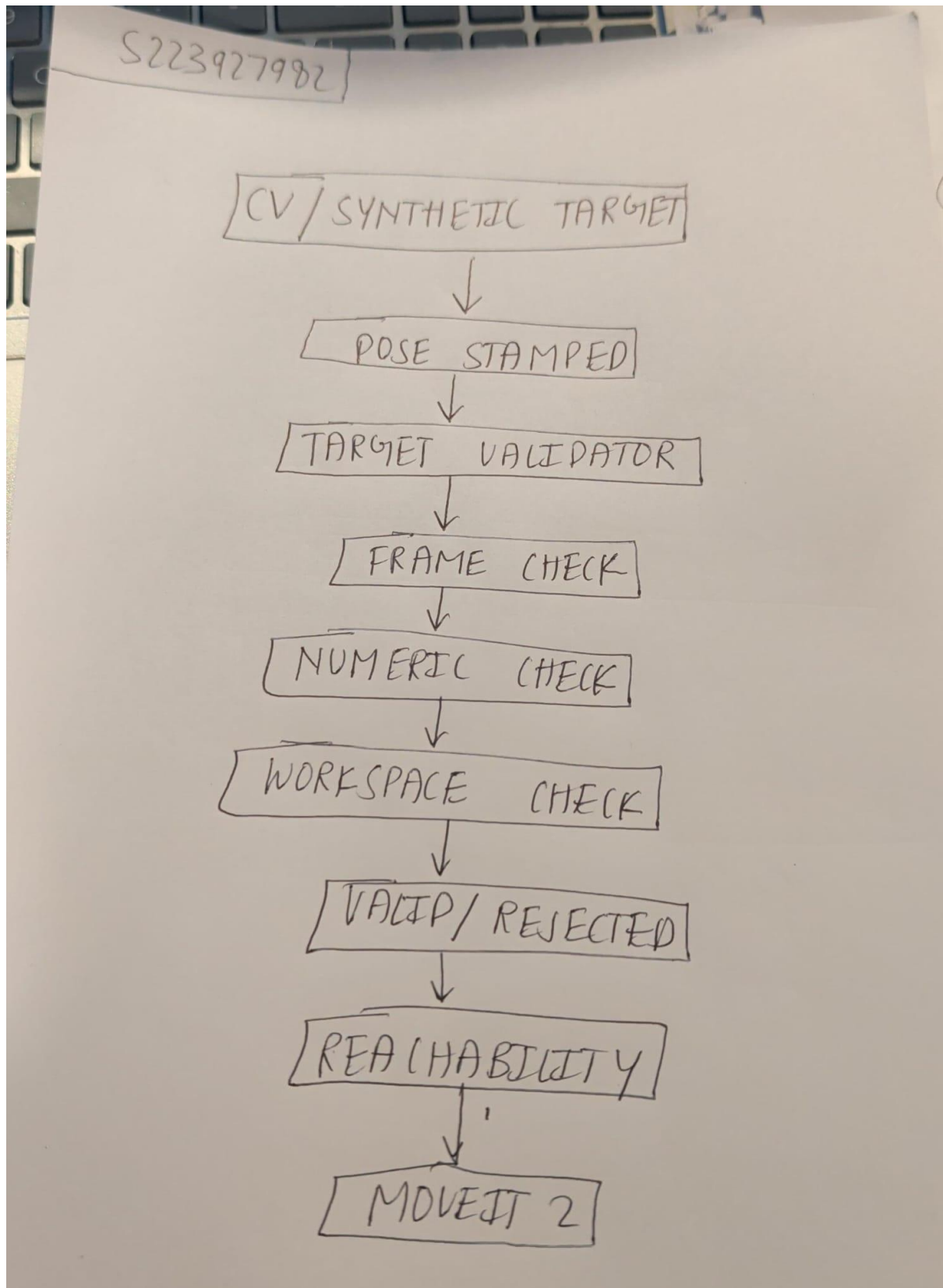
11. SPRINT 2 TECHNICAL CONTRIBUTION

My current Sprint 2 contribution is:

ROS 2 TARGET-POSE VALIDATION AND SIMULATION INTEGRATION SUPPORT

The objective is to create a reusable component between perception and robotic planning.

The intended architecture is:



12. DEVELOPMENT ENVIRONMENT

Before development began, I evaluated the available Ubuntu environment.

The system was running:

Ubuntu 22.04.5 LTS

Codename: Jammy

Initial testing showed that ROS 2 was not installed.

Commands such as:

```
printenv ROS_DISTRO
```

returned no ROS distribution, and:

```
ls /opt/ros
```

confirmed that the ROS installation directory did not exist.

This initial environment investigation prevented development from continuing under incorrect assumptions.

13. ROS 2 HUMBLE INSTALLATION AND CONFIGURATION

ROS 2 Humble was selected because it is appropriate for Ubuntu 22.04.

After installation, the environment was sourced using:

```
source /opt/ros/humble/setup.bash
```

The setup was also added to the shell configuration so that ROS 2 would automatically load when a terminal is opened.

This provided the foundation required for ROS 2 Python development.

14. ROS 2 WORKSPACE CREATION

I created a dedicated development workspace:

```
battery_ws
```

with the standard ROS 2 structure:

```
battery_ws/
```

```
|
```

```
|— src/
```

```
|— build/
```

```
|— install/
```

└─ log/

Within this workspace I created the ROS 2 package:

battery_target_validation

The package uses:

ament_python

and depends primarily upon:

roscpp

geometry_msgs

This modular package design ensures that the target-validation functionality remains separate from other project components.

15. CURRENT SOFTWARE ARCHITECTURE

Two ROS 2 nodes have currently been implemented.

Node 1

fake_target_publisher

Purpose:

Simulate future Computer Vision output.

Node 2

target_validator

Purpose:

Receive and validate target positions.

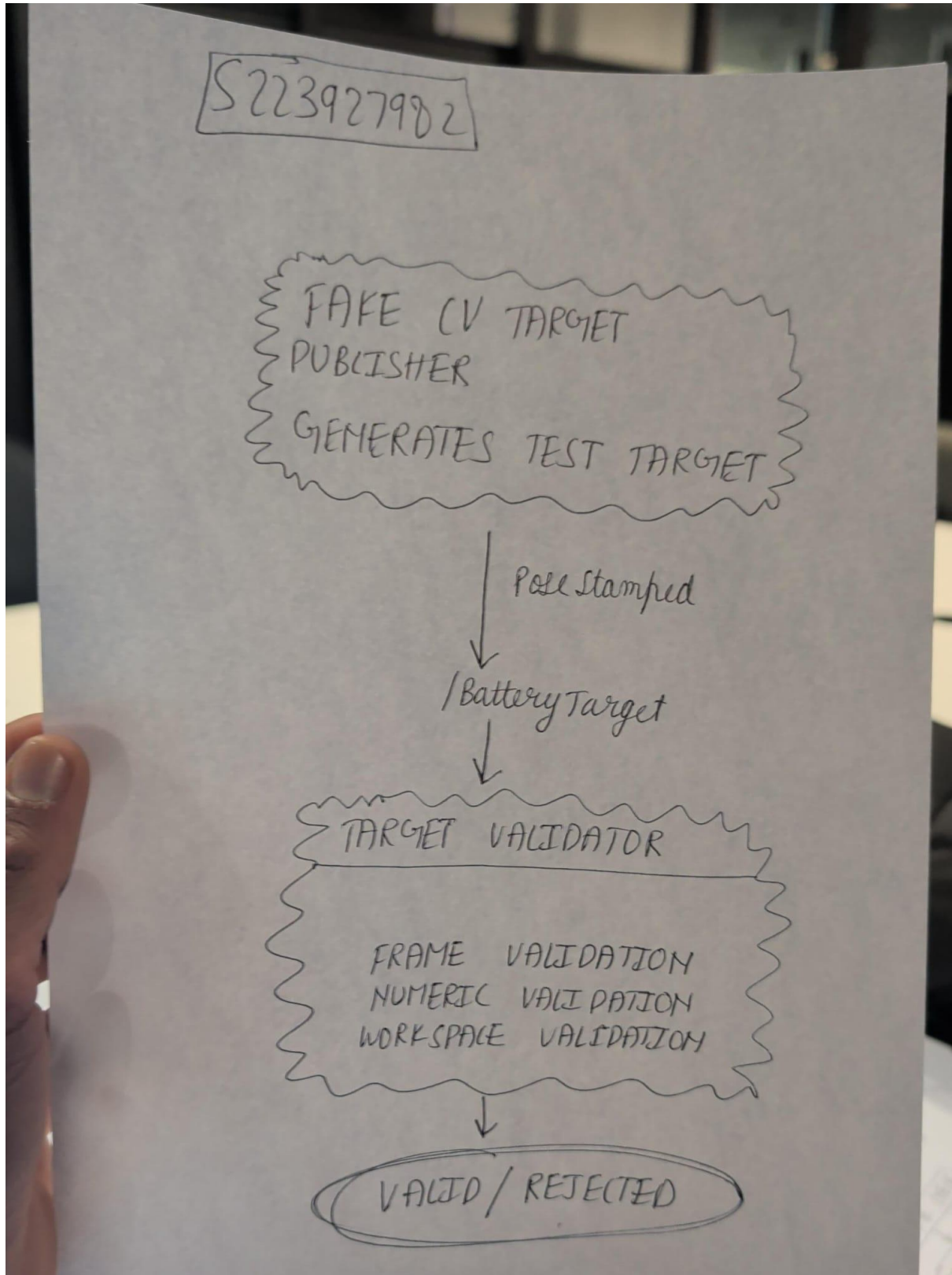
Communication occurs using:

geometry_msgs/PoseStamped

over:

/battery_target

The architecture is:



16. WHY A FAKE CV PUBLISHER WAS DEVELOPED

The final Computer Vision pipeline is still under development.

Rather than waiting for another team before starting validation work, I implemented a synthetic publisher that generates controlled target information.

This provides several advantages:

- independent development;
- repeatable testing;
- controlled test data;
- easier debugging;
- reduced dependency on external components;
- preparation for later CV integration.

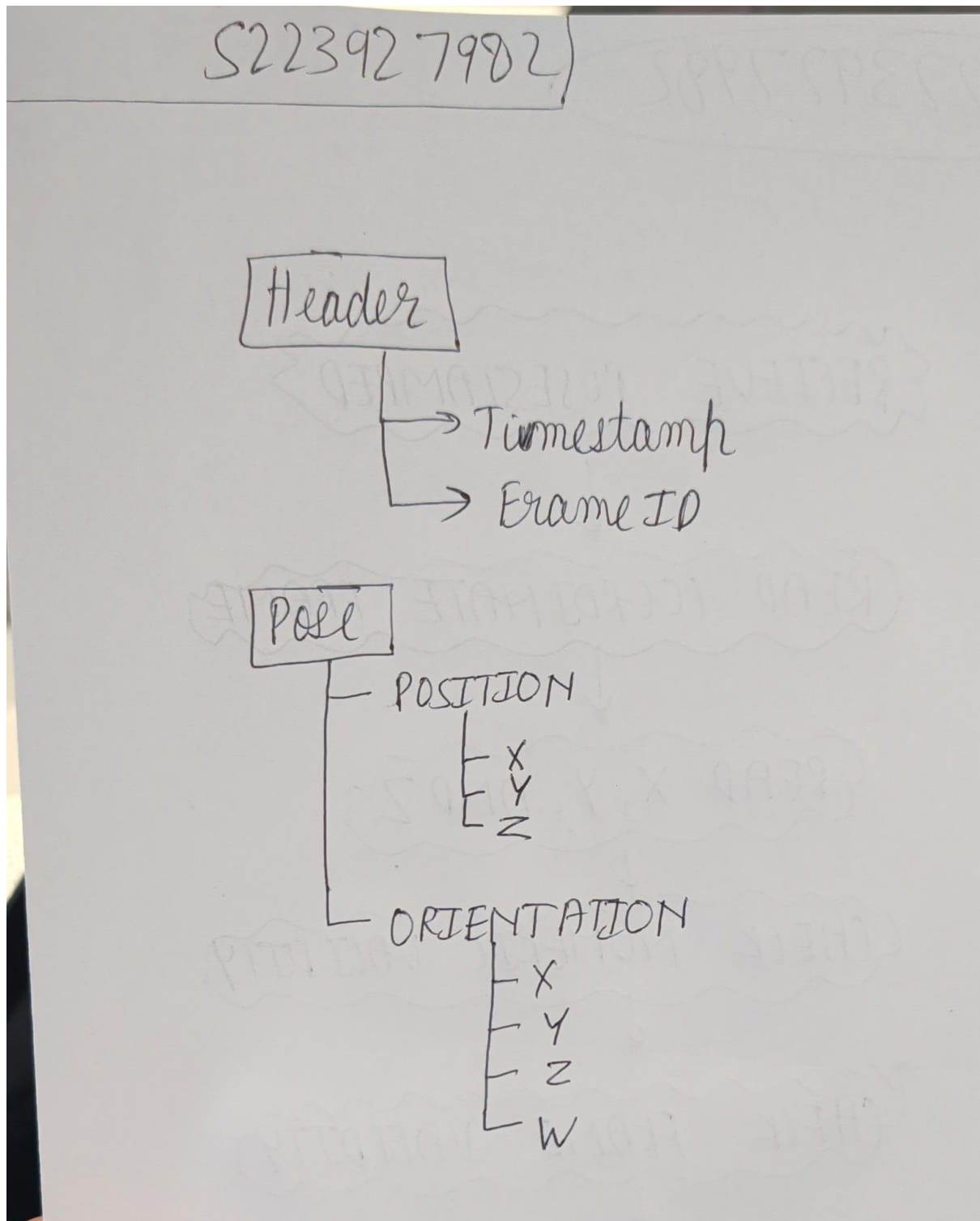
This follows good modular engineering practice because components can be developed and tested independently.

17. POSESTAMPED MESSAGE STRUCTURE

The implementation uses:

`geometry_msgs/msg/PoseStamped`

The message contains:



Using PoseStamped is useful because a coordinate without its reference frame can be ambiguous.

For example:

$x = 0.4$

$y = 0.1$

$z = 0.05$

has limited meaning unless the coordinate system is known.

18. CURRENT TEST TARGET

The synthetic publisher currently generates:

Frame = world

$X = 0.40 \text{ m}$

$Y = 0.10 \text{ m}$

$Z = 0.05 \text{ m}$

The node publishes the target repeatedly through:

/battery_target

The terminal confirmed successful operation:

Fake CV Target Publisher started

Published fake battery target:

$x=0.40, y=0.10, z=0.05$

19. TARGET VALIDATOR

The target_validator subscribes to:

/battery_target

and extracts:

frame_id

x

y

z

The current validation sequence is:

S223927982

RECEIVE POSESTAMPED



READ COORDINATE FRAME



READ X, Y, AND Z



CHECK NUMERIC VALIDITY



CHECK FRAME VALIDITY



CHECK WORKSPACE LIMITS



RETURN VALID / REJECTED

20. COORDINATE-FRAME VALIDATION

The prototype currently accepts expected frames such as:

world

base_link

The first successful test used:

frame_id = world

This is important because future CV and Robotics integration requires a clearly defined coordinate convention.

The final project must eventually specify:

- source frame;
- target frame;
- coordinate origin;
- X direction;
- Y direction;
- Z direction;
- units;
- TF transformation responsibility.

21. NUMERICAL VALIDATION

The validator checks that X, Y and Z contain valid finite numbers.

This prevents malformed information such as:

NaN

Infinity

from progressing into downstream robotic components.

This follows defensive software-engineering principles by validating information at subsystem boundaries.

22. WORKSPACE VALIDATION

The validator currently contains preliminary workspace limits.

These limits are currently development values used to prove the validation concept.

They are not being presented as final Kinova workspace parameters.

The final values will need to be aligned with:

- Gazebo scene dimensions;
- robot base position;
- Kinova Gen3 workspace;
- phone placement;
- sorting-bin locations.

23. FIRST SUCCESSFUL END-TO-END TEST

The system was tested using two independent terminals.

The validator was executed using:

```
ros2 run battery_target_validation target_validator
```

The publisher was executed using:

```
ros2 run battery_target_validation fake_target_publisher
```

The publisher produced:

Published fake battery target:

x=0.40, y=0.10, z=0.05

The validator successfully received:

Received target:

frame=world,

x=0.400,

y=0.100,

z=0.050

and generated:

TARGET VALID

This confirms successful communication between two independently running ROS 2 nodes.

24. WHAT HAS BEEN DEMONSTRATED SO FAR

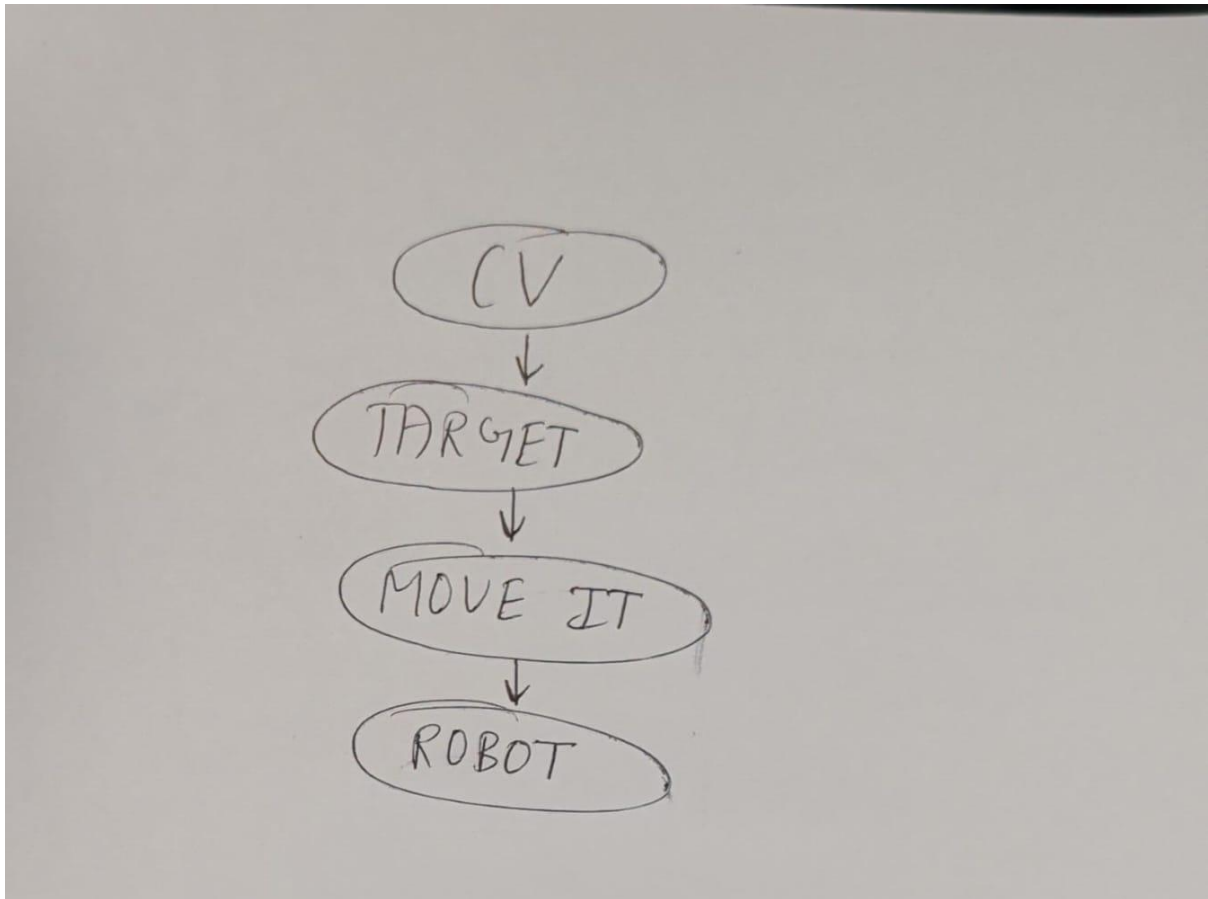
The implementation currently demonstrates:

- ROS 2 Humble environment configuration;
- Ubuntu/Linux development;
- Python ROS 2 programming;
- creation of a custom ROS 2 package;
- publisher development;
- subscriber development;
- publish/subscribe communication;
- PoseStamped message handling;
- coordinate-frame awareness;
- workspace validation;
- numerical validation;
- synthetic subsystem testing;
- modular software architecture.

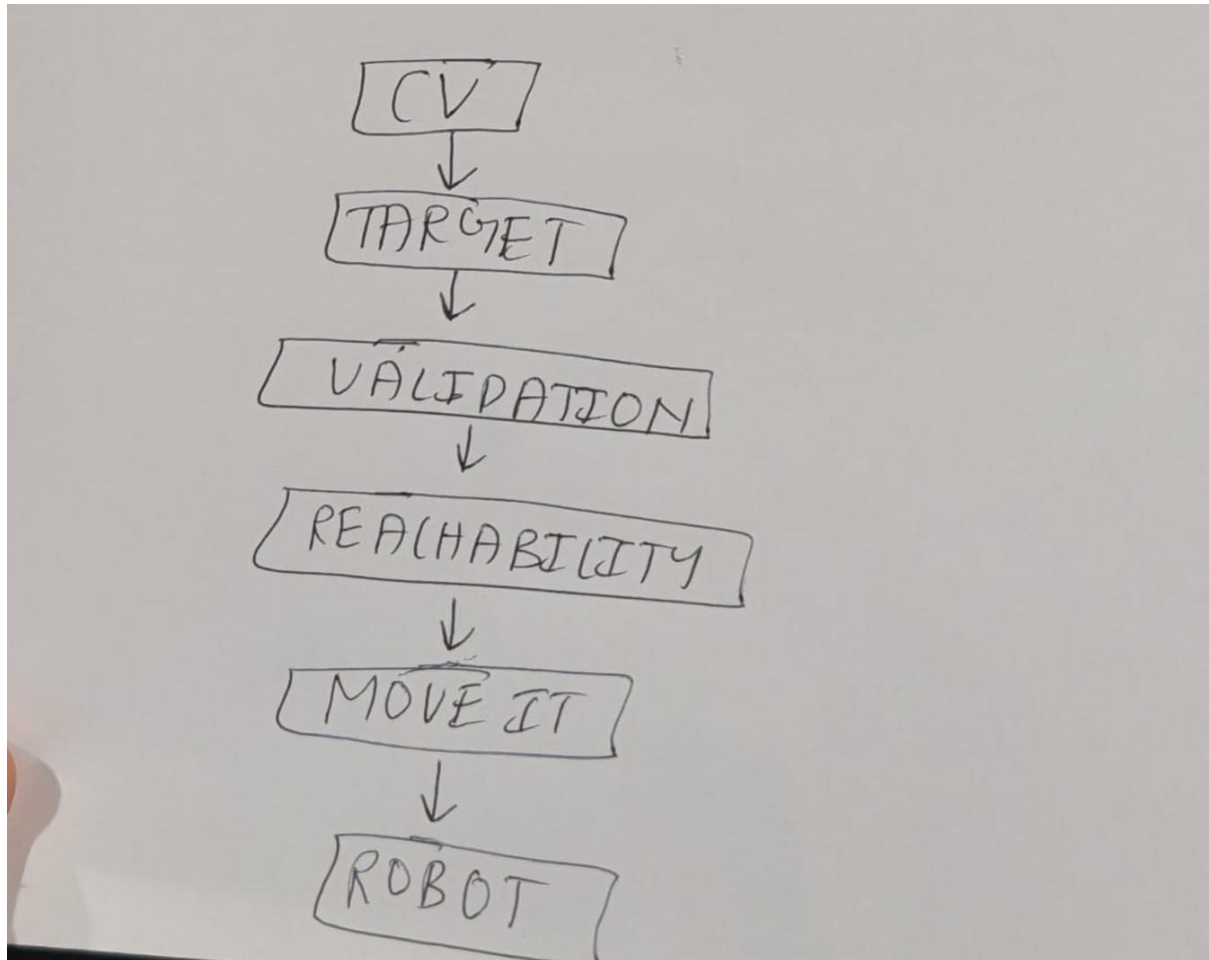
Most importantly, this is an implementation derived directly from the earlier engineering investigation.

25. Engineering Significance

Without an intermediate validation layer, the system could effectively become:



The architecture I am developing instead introduces:



This introduces a safeguard between perception and robotic execution.

The validator provides a location where malformed, incorrect or clearly unusable targets can be rejected before being passed towards motion planning.

26. CURRENT LIMITATIONS

The implementation is still in progress.

Current limitations include:

- synthetic input rather than final CV input;
- preliminary workspace boundaries;
- no full TF2 transformation implementation yet;
- no formal validated-output topic yet;
- no complete negative test suite yet;
- no CSV test logging yet;
- no direct Gazebo integration yet;
- no direct MoveIt! integration yet;

- no final reachability integration yet.

These limitations define the remaining development work.

27. NEXT STAGE – NEGATIVE TESTING

The next immediate milestone is systematic validation testing.

Planned cases include:

TEST	INPUT CONDITION	EXPECTED RESULT
T01	NORAMAL TARGET	VALID
T02	X outside workspace	REJECT
T03	Y outside workspace	REJECT
T04	Z below workspace	REJECT
T05	Z above workspace	REJECT
T06	Wrong frame	REJECT
T07	NaN coordinate	REJECT
T08	Boundary coordinate	Correct boundary behaviour

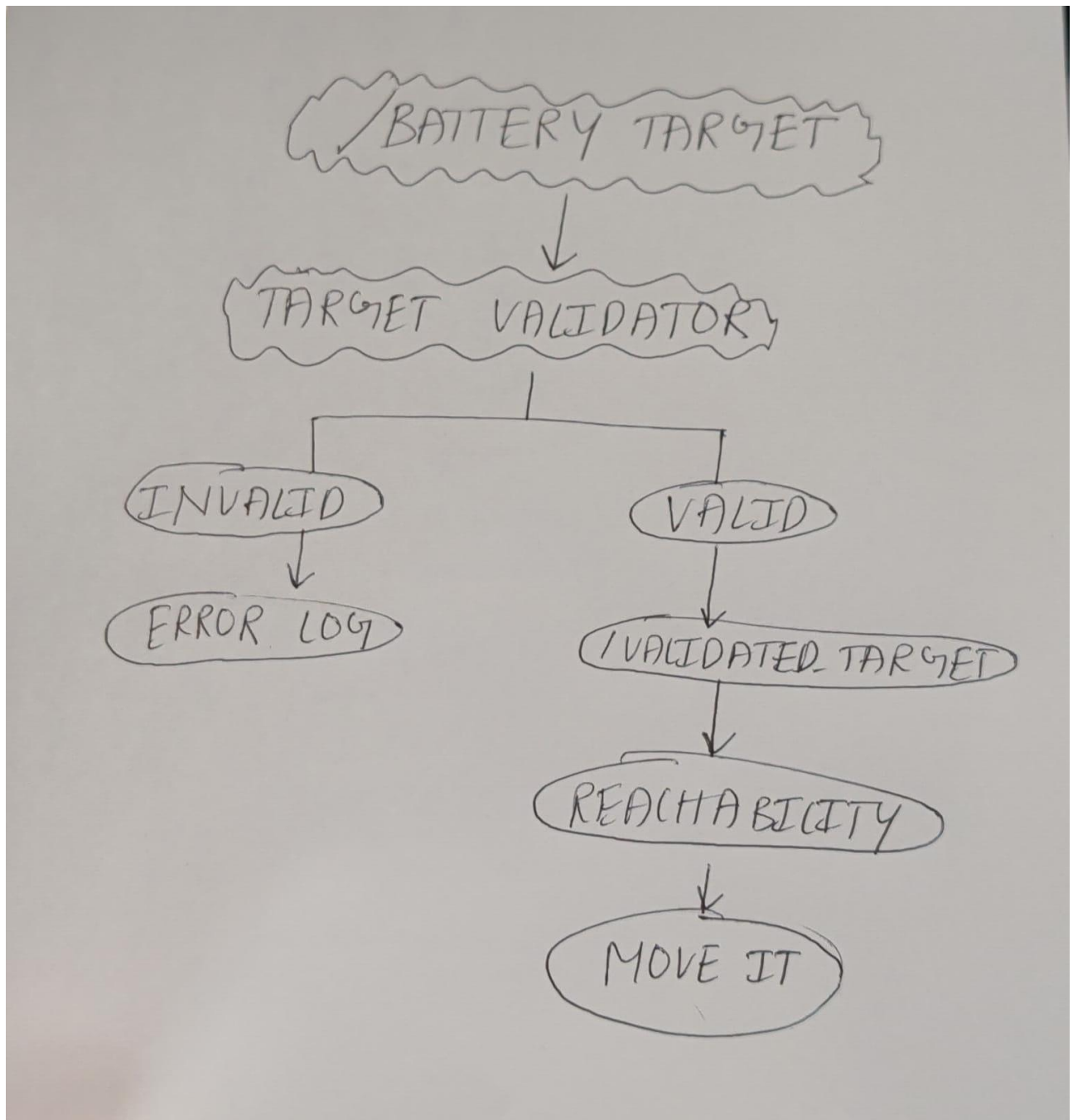
This will transform the current proof-of-concept into a repeatable testable component.

28. FUTURE VALIDATED-OUTPUT ARCHITECTURE

The current validator reports its decision through terminal logs.

The future version should publish valid targets onward.

Proposed architecture:



This would convert the validator from a diagnostic tool into a reusable system component.

29. COORDINATE INTERFACE SPECIFICATION

Another planned deliverable is a formal interface agreement defining the hand-off between CV, Simulation and Robotics.

The specification should include:

- message type;

- topic name;
- coordinate frame;
- units;
- axis directions;
- timestamp handling;
- transformation responsibility;
- acceptable bounds;
- invalid-target behaviour.

This will reduce integration ambiguity between teams.

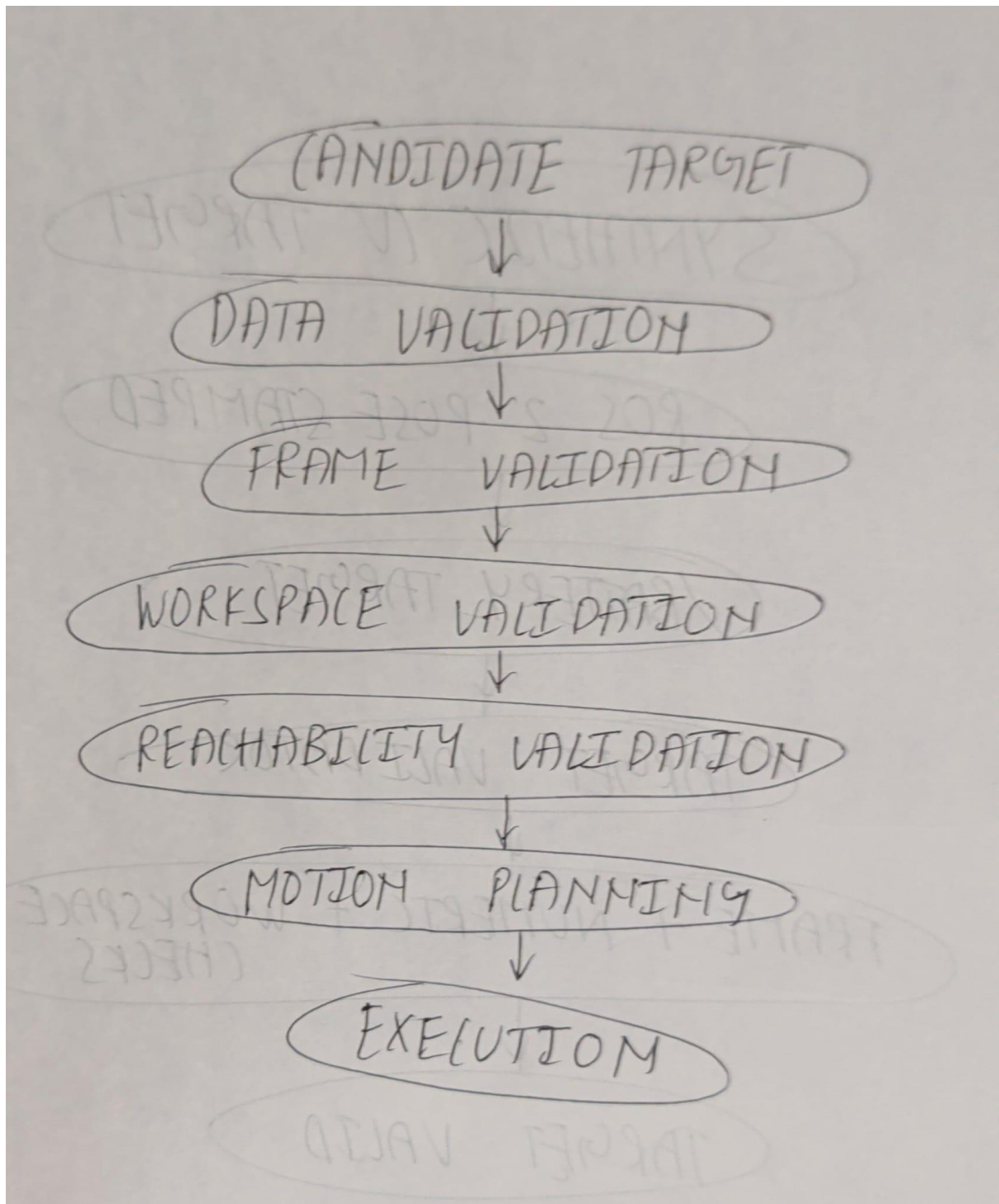
30. REACHABILITY INTEGRATION

Workspace bounds alone cannot guarantee reachability.

A point may lie inside a rectangular workspace but still be unreachable due to:

- robot geometry;
- joint limits;
- obstacles;
- orientation requirements;
- singularities.

Therefore the longer-term validation sequence should become:



Integration with the team's existing Role 3 reachability work will therefore be explored.

31. LOGGING AND TEST EVIDENCE

A later version will record validation information such as:

timestamp

frame_id

x

y

z

result

reason

Example:

12:30:05, world, 0.40, 0.10, 0.05, VALID, -

12:30:07, world, 2.00, 0.10, 0.05, REJECTED, X_OUT_OF_RANGE

This will provide repeatable evidence of system behaviour and make test results easier to review.

32. RISK CONSIDERATIONS

The implementation directly addresses several risks identified earlier.

Incorrect localisation

Mitigation:

Validate incoming target values before planning.

Incorrect coordinate frame

Mitigation:

Check frame_id before accepting a pose.

Invalid numerical values

Mitigation:

Reject malformed coordinates.

Unreachable targets

Mitigation:

Introduce workspace and future reachability validation.

Integration inconsistency

Mitigation:

Define an explicit ROS 2 interface contract.

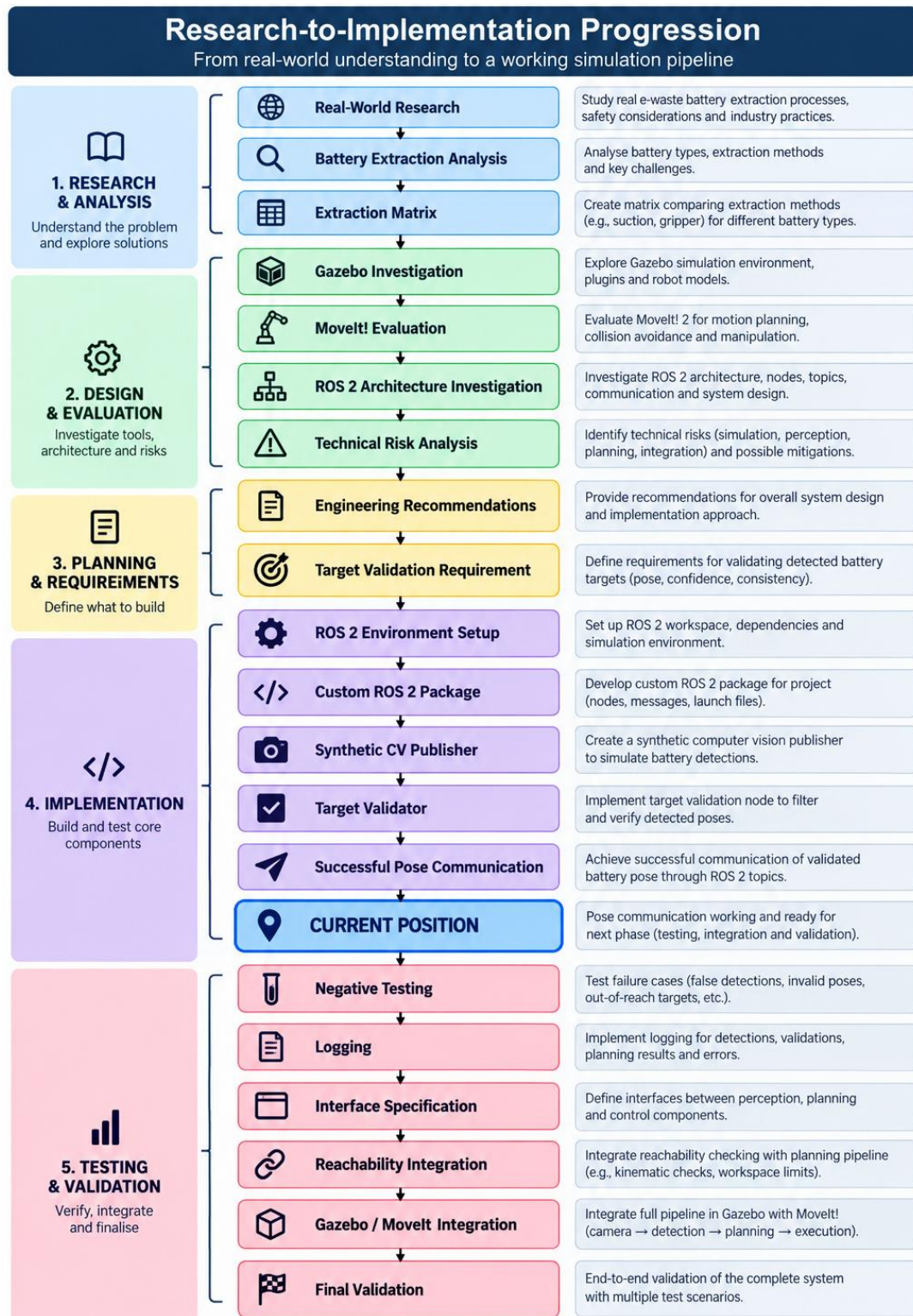
Dependency on unfinished CV work

Mitigation:

Use a synthetic publisher for independent testing.

33. RESEARCH-TO-IMPLEMENTATION PROGRESSION

The complete contribution now follows:



This progression demonstrates that the implementation was not developed arbitrarily. It emerged directly from the engineering analysis undertaken earlier in the trimester.

34. CONTRIBUTION TO THE WIDER PROJECT

The work is designed to support several teams.

Simulation Team

Provides a validation mechanism for target locations entering the simulation workflow.

Computer Vision Team

Provides a clearly defined target-message interface for future detection output.

Robotics Team

Reduces the possibility of obviously invalid target information reaching reachability or motion planning.

Future Cohorts

Provides reusable technical documentation and ROS 2 code that can be extended rather than rebuilt from the beginning.

35. SKILLS DEMONSTRATED

Through this combined research and implementation work, I have demonstrated progress in:

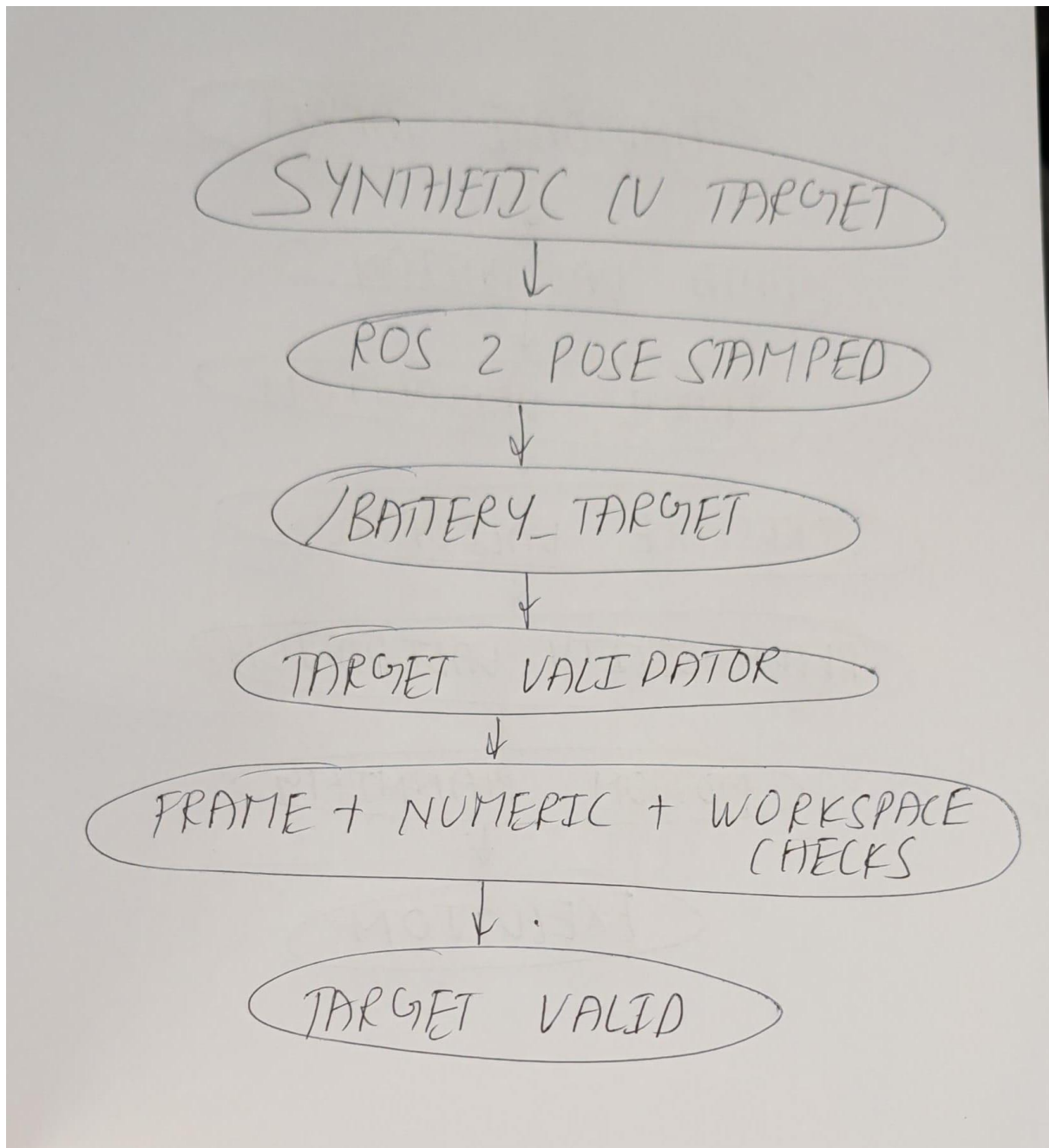
- software engineering analysis;
- technical research;
- Linux development;
- Python;
- ROS 2 Humble;
- rclpy;
- publishers and subscribers;
- ROS topics;
- ROS message types;
- simulation architecture;
- motion-planning investigation;
- coordinate-frame reasoning;

- systems integration;
- defensive programming;
- technical testing;
- risk analysis;
- technical documentation;
- independent engineering learning.

36. CURRENT STATUS

At the current stage, the research and initial implementation foundations are complete.

The current working functionality is:



The first positive integration test has been completed successfully.

The next priority is to move from a successful positive test towards comprehensive negative testing and system integration.

37. FUTURE WORK

The remaining implementation work will focus on:

1. developing automated valid/invalid test cases;
2. improving rejection reporting;

3. adding structured logging;
4. defining the coordinate interface;
5. publishing validated targets;
6. aligning workspace limits with the actual simulation;
7. integrating with the team's reachability work;
8. testing against CV-produced targets when available;
9. investigating TF2 coordinate transformations;
10. evaluating Gazebo and MoveIt! integration;
11. producing a final demonstration;
12. documenting GitHub commits, test results and technical evidence.

38. OVERALL REFLECTION

The most significant progression in my contribution has been the transition from **research into implementation**.

During the first sprint, I concentrated on understanding the problem properly rather than immediately developing software without sufficient engineering analysis. This resulted in a detailed investigation into realistic battery extraction, simulation limitations, motion planning, integration risks and system architecture.

During Sprint 2, I began applying those findings practically by developing a ROS 2 target-validation component. The successful publisher/subscriber communication demonstrates that the first implementation milestone has been achieved.

This process strengthened my understanding that software engineering within robotics involves more than writing code. It requires understanding the physical problem, analysing system constraints, defining interfaces, identifying risks, selecting appropriate technologies, implementing modular software and validating behaviour before integration.

39. CONCLUSION

This technical contribution combines engineering research, systems analysis and practical ROS 2 software development to support the E-Waste Battery Extraction project.

The initial research established that realistic battery extraction requires significantly greater consideration than basic robotic pick-and-place behaviour. Through investigation of smartphone repair procedures, Gazebo physics, MoveIt! planning and ROS 2 architecture, I developed recommendations for a practical hybrid simulation strategy and identified target validation as an important integration requirement.

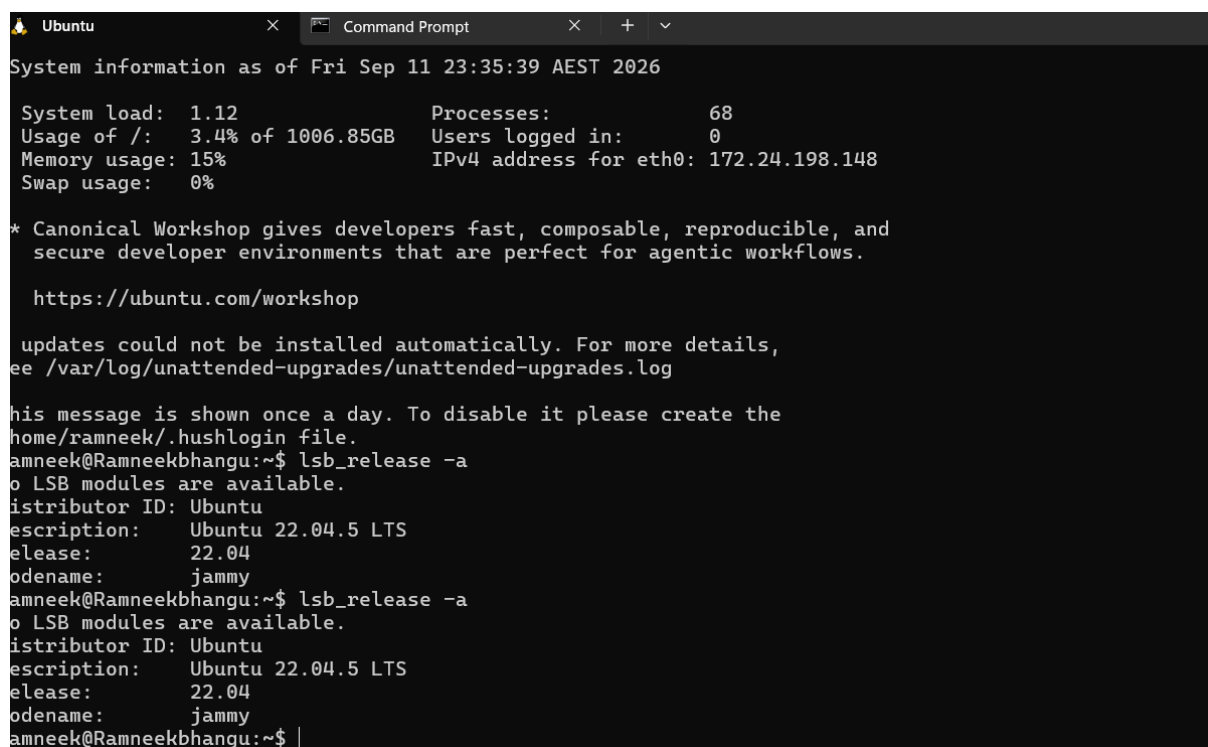
Building upon these findings, I successfully established a ROS 2 Humble development environment, created a dedicated ROS 2 package and implemented two independent Python

nodes: a synthetic Computer Vision target publisher and a target-pose validation subscriber. The first end-to-end test confirmed successful transmission and validation of a PoseStamped battery target.

The project now has a working foundation for further validation, testing and integration. The remaining work will extend this component through negative testing, structured logging, coordinate-interface definition, reachability support and eventual connection with the wider Simulation, Computer Vision and Robotics pipeline.

Overall, this contribution demonstrates a clear progression from **engineering investigation** → **design** → **implementation** → **validation** → **integration**, while creating a technical component and documentation that can continue to support the project beyond my individual work.

EVIDENCE SCREENSHOTS:-

A screenshot of a terminal window titled 'Ubuntu' and 'Command Prompt'. The terminal displays system information as of Friday, September 11, 2026, at 23:35:39 AEST. It shows system load (1.12), usage of / (3.4% of 1006.85GB), memory usage (15%), swap usage (0%), processes (68), users logged in (0), and IPv4 address for eth0 (172.24.198.148). It also includes a Canonical Workshop advertisement and a message about unattended-upgrades. Finally, it shows the output of the 'lsb_release -a' command, indicating Ubuntu 22.04.5 LTS with the codename 'jammy'.

```
System information as of Fri Sep 11 23:35:39 AEST 2026

System load:  1.12                Processes:            68
Usage of /:   3.4% of 1006.85GB   Users logged in:     0
Memory usage: 15%                IPv4 address for eth0: 172.24.198.148
Swap usage:   0%

* Canonical Workshop gives developers fast, composable, reproducible, and
  secure developer environments that are perfect for agentic workflows.

  https://ubuntu.com/workshop

updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log

This message is shown once a day. To disable it please create the
/home/ramneek/.hushlogin file.
ramneek@Ramneekbhangu:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.5 LTS
Release:        22.04
Codename:       jammy
ramneek@Ramneekbhangu:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.5 LTS
Release:        22.04
Codename:       jammy
ramneek@Ramneekbhangu:~$ |
```

RAMNEEK SINGH BHANGU(S223927982)

```
Ubuntu x Command Prompt x + v
Get:347 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libqt5webkit5-dev amd64 5.212.0~alpha4-15ubuntu1 [54.7 kB]
Get:348 http://archive.ubuntu.com/ubuntu jammy/main amd64 sgml-base all 1.30 [12.5 kB]
Get:349 http://archive.ubuntu.com/ubuntu jammy/main amd64 libtcl8.6 amd64 8.6.12+dfsg-1build1 [990 kB]
Get:350 http://archive.ubuntu.com/ubuntu jammy/main amd64 libtk8.6 amd64 8.6.12-1build1 [784 kB]
Get:351 http://archive.ubuntu.com/ubuntu jammy/main amd64 tk8.6-blt2.5 amd64 2.5.3+dfsg-4.1build2 [643 kB]
Get:352 http://archive.ubuntu.com/ubuntu jammy/main amd64 blt amd64 2.5.3+dfsg-4.1build2 [4,838 B]
Get:353 http://archive.ubuntu.com/ubuntu jammy/main amd64 java-common all 0.72build2 [6,782 B]
Get:354 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libpcsclite1 amd64 1.9.5-3ubuntu1 [19.8 kB]
Get:355 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 openjdk-11-jre-headless amd64 11.0.32+9-1ubuntu1~22.04
[42.6 MB]
Get:356 http://archive.ubuntu.com/ubuntu jammy/main amd64 default-jre-headless amd64 2:1.11-72build2 [3,042 B]
Get:357 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ca-certificates-java all 20190909ubuntu1.3 [12.3 kB]
Get:358 http://archive.ubuntu.com/ubuntu jammy/universe amd64 catch2 amd64 2.13.8-1 [490 kB]
Get:359 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libtinyxml2-9 amd64 9.0.0+dfsg-3 [32.5 kB]
Get:360 http://archive.ubuntu.com/ubuntu jammy/universe amd64 cppcheck amd64 2.7-1 [2,016 kB]
Get:361 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgif7 amd64 5.1.9-2ubuntu0.3 [34.3 kB]
Get:362 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 openjdk-11-jre amd64 11.0.32+9-1ubuntu1~22.04 [214 kB]
Get:363 http://archive.ubuntu.com/ubuntu jammy/main amd64 default-jre amd64 2:1.11-72build2 [896 B]
Get:364 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 openjdk-11-jdk-headless amd64 11.0.32+9-1ubuntu1~22.04
[73.7 MB]
Get:365 http://archive.ubuntu.com/ubuntu jammy/main amd64 default-jdk-headless amd64 2:1.11-72build2 [942 B]
Get:366 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 openjdk-11-jdk amd64 11.0.32+9-1ubuntu1~22.04 [1,350 k
B]
Get:367 http://archive.ubuntu.com/ubuntu jammy/main amd64 default-jdk amd64 2:1.11-72build2 [908 B]
Get:368 http://archive.ubuntu.com/ubuntu jammy/main amd64 mysql-common all 5.8+1.0.8 [7,212 B]
Get:369 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libmysqlclient21 amd64 8.0.46-0ubuntu0.22.04.4 [1,336
kB]
Get:370 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libssl-dev amd64 3.0.2-0ubuntu1.29 [2,376 kB]
Get:371 http://archive.ubuntu.com/ubuntu jammy/main amd64 libzstd-dev amd64 1.4.8+dfsg-3build1 [401 kB]
39% [Waiting for headers] 800 kB/s 6min 20s|
```

```
Ubuntu x Command Prompt x + v
options:
  -h, --help                show this help message and exit
  --use-python-default-buffering
                           Do not force line buffering in stdout and instead use the python default buffering, which
                           might be affected by PYTHONUNBUFFERED/-u and depends on whatever stdout is interactive or not

Commands:
  action                    Various action related sub-commands
  bag                       Various rosbag related sub-commands
  component                 Various component related sub-commands
  daemon                   Various daemon related sub-commands
  doctor                   Check ROS setup and other potential issues
  interface                 Show information about ROS interfaces
  launch                   Run a launch file
  lifecycle                 Various lifecycle related sub-commands
  multicast                 Various multicast related sub-commands
  node                     Various node related sub-commands
  param                    Various param related sub-commands
  pkg                      Various package related sub-commands
  plugin                   Various plugin related sub-commands
  run                      Run a package specific executable
  security                 Various security related sub-commands
  service                  Various service related sub-commands
  topic                    Various topic related sub-commands
  wtf                      Use 'wtf' as alias to 'doctor'

  Call 'roslaunch <command> -h' for more detailed usage.
ramneek@Ramneekbhangu:~$ printenv ROS_DISTRO
humble
ramneek@Ramneekbhangu:~$ |
```

RAMNEEK SINGH BHANGU(S223927982)

```
ramneek@Ramneekbhangu: ~$ cat CTEST_COMMAND
The full path to the CTest executable
ramneek@Ramneekbhangu: ~$ cat POWERSHELL_COMMAND
The full path to the PowerShell executable

for more help and usage tips, see https://colcon.readthedocs.io
ramneek@Ramneekbhangu: ~$ mkdir -p ~/battery_ws/src
cd ~/battery_ws/src
ramneek@Ramneekbhangu: ~/battery_ws/src$ ros2 pkg create --build-type ament_python battery_target_validation --dependencies
rclpy geometry_msgs
going to create a new package
package name: battery_target_validation
destination directory: /home/ramneek/battery_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['ramneek <ramneek@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_python
dependencies: ['rclpy', 'geometry_msgs']
creating folder ./battery_target_validation
creating ./battery_target_validation/package.xml
creating source folder
creating folder ./battery_target_validation/battery_target_validation
creating ./battery_target_validation/setup.py
creating ./battery_target_validation/setup.cfg
creating folder ./battery_target_validation/resource
creating ./battery_target_validation/resource/battery_target_validation
creating ./battery_target_validation/battery_target_validation/__init__.py
creating folder ./battery_target_validation/test
creating ./battery_target_validation/test/test_copyright.py
creating ./battery_target_validation/test/test_flake8.py
```

```
MIT
MIT-0
ramneek@Ramneekbhangu: ~/battery_ws/src$ ros2 pkg create --build-type ament_python battery_target_validation --dependencies
rclpy geometry_msgs
Aborted!
The directory already exists: ./battery_target_validation
Either remove the directory or choose a different destination directory or package name
ramneek@Ramneekbhangu: ~/battery_ws/src$ cd ~/battery_ws
colcon build
Starting >>> battery_target_validation
Finished <<< battery_target_validation [0.83s]

Summary: 1 package finished [1.37s]
ramneek@Ramneekbhangu: ~/battery_ws$ source install/setup.bash
ramneek@Ramneekbhangu: ~/battery_ws$ echo "source ~/battery_ws/install/setup.bash" >> ~/.bashrc
ramneek@Ramneekbhangu: ~/battery_ws$ cd ~/battery_ws/src/battery_target_validation/battery_target_validation
ramneek@Ramneekbhangu: ~/battery_ws/src/battery_target_validation/battery_target_validation$ nano target_validator.py
ramneek@Ramneekbhangu: ~/battery_ws/src/battery_target_validation/battery_target_validation$ cd ~/battery_ws/src/battery
target_validation
ramneek@Ramneekbhangu: ~/battery_ws/src/battery_target_validation$ nano setup.py
ramneek@Ramneekbhangu: ~/battery_ws/src/battery_target_validation$ cd ~/battery_ws
colcon build --packages-select battery_target_validation
Starting >>> battery_target_validation
Finished <<< battery_target_validation [0.72s]

Summary: 1 package finished [1.13s]
ramneek@Ramneekbhangu: ~/battery_ws$ |
```

TEST 1:-

```
Summary: 1 package finished [1.13s]
ramneek@Ramneekbhangu: ~/battery_ws$ source install/setup.bash
ramneek@Ramneekbhangu: ~/battery_ws$ source ~/battery_ws/install/setup.bash
ros2 run battery_target_validation target_validator
[INFO] [1789137801.578007157] [target_validator]: Battery Target Validator started
```

RAMNEEK SINGH BHANGU(S223927982)

```
ramneek@Ramneekbhangu: ~$ source install/setup.bash
Starting >>> battery_target_validation
Finished <<< battery_target_validation [0.72s]

Summary: 1 package finished [1.13s]
ramneek@Ramneekbhangu:~/battery_ws$ source install/setup.bash
ramneek@Ramneekbhangu:~/battery_ws$ source ~/battery_ws/install/setup.bash
ros2 run battery_target_validation target_validator
[INFO] [1789137801.578007157] [target_validator]: Battery Target Validator started
[INFO] [1789137856.175647454] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137856.175968159] [target_validator]: TARGET VALID
[INFO] [1789137856.176560784] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137856.176848565] [target_validator]: TARGET VALID
[INFO] [1789137857.797416623] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137857.798648192] [target_validator]: TARGET VALID
[INFO] [1789137859.796541471] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137859.796977996] [target_validator]: TARGET VALID
[INFO] [1789137861.809637242] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137861.810332692] [target_validator]: TARGET VALID
[INFO] [1789137863.796135562] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137863.796770100] [target_validator]: TARGET VALID
[INFO] [1789137865.797591414] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137865.798550377] [target_validator]: TARGET VALID
[INFO] [1789137867.797488368] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137867.798033719] [target_validator]: TARGET VALID
[INFO] [1789137869.798540516] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137869.799131803] [target_validator]: TARGET VALID
[INFO] [1789137871.805440000] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137871.805885555] [target_validator]: TARGET VALID
[INFO] [1789137873.797636624] [target_validator]: Received target: frame=world, x=0.400, y=0.100, z=0.050
[INFO] [1789137873.798356860] [target_validator]: TARGET VALID
```

```
This message is shown once a day. To disable it please create the
/home/ramneek/.hushlogin file.
ramneek@Ramneekbhangu:~$ source ~/battery_ws/install/setup.bash
ros2 run battery_target_validation fake_target_publisher
[INFO] [1789137851.802510784] [fake_target_publisher]: Fake CV Target Publisher started
[INFO] [1789137853.796985845] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137855.796597777] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137857.795980729] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137859.795506409] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137861.808222350] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137863.795535106] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137865.796316742] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137867.796018496] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137869.797159416] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137871.804602115] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137873.796470747] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137875.795668462] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137877.795656387] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137879.795045984] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137881.806360281] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137883.795437571] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137885.794859520] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137887.795238043] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137889.795829303] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137891.797525936] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137893.796370813] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137895.795155410] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137897.795606104] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
[INFO] [1789137899.795062602] [fake_target_publisher]: Published fake battery target: x=0.40, y=0.10, z=0.05
```

Error > RAMNEEK SINGH - Deakin University > Desktop > SIT378-TPRJ > battery_target_validation >

Name	Status	Date modified	Type	Size
battery_target_validation	✓	12/09/2026 3:08 AM	File folder	
resource	✓	12/09/2026 3:08 AM	File folder	
package.xml	✓	12/09/2026 3:10 AM	Microsoft Edge HT...	1 KB
README.md	✓	12/09/2026 3:11 AM	Markdown Source ...	1 KB
setup.cfg	✓	12/09/2026 3:09 AM	Configuration Sou...	1 KB
setup.py	✓	12/09/2026 3:09 AM	Python Source File	2 KB

RAMNEEK SINGH BHANGU(S223927982)

Name	Status	Date modified	Type	Size
 fake_target_publisher.py	✔	12/09/2026 3:09 AM	Python Source File	2 KB
 init_.py	✔	12/09/2026 3:07 AM	Python Source File	0 KB
 target_validator.py	✔	12/09/2026 3:08 AM	Python Source File	3 KB

VIDEO DEMONSTRATION:-

<https://youtu.be/GWSTtUEg27Q>

GITHUB:-

[Ramneek-commits/battery_target_validation](#)